

Practice Exercise 14: S/KEY Identification Protocol

Context

We are implementing the **S/KEY identification protocol** (based on Lamport's Hash Chain). Unlike TOTP, which relies on synchronized clocks, S/KEY relies on a mathematical "chain" of hashes consumed in reverse order.

This protocol involves two stateful entities:

1. **The Prover (Client):** Holds the secret generation parameters.
2. **The Verifier (Server):** Holds the last successful verification key.

The Logic (Formal Definition)

Let $H(x)$ be a hash function (SHA-256). Let $H^{(n)}(x)$ denote applying the hash function n times recursively.

1. **Setup phase:**
 - Prover generates a secret seed k .
 - They agree on a sequence length N .
 - Verifier receives and stores the initial public key: $vk = H^{(N+1)}(k)$.
 - Prover initializes their state: $sk = (k, N)$.
2. **Identification phase (The Login):**
 - **Prover:** To prove identity at step i , computes $t = H^{(i)}(k)$.
 - **Prover:** Updates internal state: $i \leftarrow i - 1$. Sends t to Verifier.
 - **Verifier:** Checks if $H(t) == vk$.
 - If **True:** The token is valid (it is the pre-image of the current lock). Verifier updates $vk \leftarrow t$ and outputs "YES".
 - If **False:** Outputs "NO".
3. **Reseeding (Voluntary Key Rotation):**
 - If the Prover runs out of tokens (or wants to change keys), they generate a new seed k_{new} .
 - To authorize this change, they must send a valid token from the *old* chain along with the anchor of the *new* chain.

Your Task

Implement the object-oriented simulation of the Prover and Verifier.

Required functions & Classes

You must implement **two classes** using Python's `hashlib`:

1. The Prover Class

class Prover:

def `__init__`(self, seed, count):

"""

Input:

seed (str): The secret key.

count (int): The initial counter N.

"""

`self.seed` = seed

`self.count` = count

def `get_token`(self):

"""

Logic:

1. Calculate $t = \text{Hash}^{\text{count}}(\text{seed})$.

2. Decrement `self.count` by 1.

3. Return `t`.

"""

def `prepare_reseed`(self, new_seed, new_count):

"""

Logic:

1. Generate 'auth_token' using `get_token()` (from the OLD chain).

2. Calculate 'new_anchor' = $\text{Hash}^{\text{new_count} + 1}(\text{new_seed})$.

3. Update `self.seed` and `self.count` to the NEW values.

4. Return (auth_token, new_anchor).

"""

2. The Verifier Class

class Verifier:

def `__init__`(self, initial_vk):

"""

Input:

initial_vk (str): The initial verification key $H^{(N+1)}(k)$.

"""

`self.vk` = initial_vk

def `verify`(self, token):

"""

Logic:

1. Compute $h = \text{SHA256}(\text{token})$.

2. Check if $h == \text{self.vk}$.

3. If True: Update `self.vk` = token; Return True.

4. If False: Return False.

"""

def `handle_reseed`(self, auth_token, new_anchor):

"""

Logic:

```

1. Call self.verify(auth_token).
2. If returns True:
    Overwrite self.vk = new_anchor.
    Return True.
3. Else: Return False.
"""

```

Dataset

Use `import hashlib`. Use a helper function for recursive hashing.

Test Case 1: Manual Check

Inputs:

- Seed: "hust"
- N: 2 (Short chain for debugging)

Pre-calculation (Debugging):

- H^0 = "hust"
- H^1 = SHA256("hust") =
9e732f7cb369f0f93a470df17a27c2c0be7b960371efd5f7fe953d8b5689b1e9
- H^2 = SHA256(H^1) =
27ebf7c9710b5e0b53d0a20dccfcda02c8c6c7f1cd86dd09da20d124b830f1a6 (Token for Login 1)
- H^3 = SHA256(H^2) =
79d0c423551a90dec165a623b831b07d63b60f45cd1cd409196424de3b5eb44e (Verifier Initial Key)

Expected Flow:

1. Verifier initialized with
79d0c423551a90dec165a623b831b07d63b60f45cd1cd409196424de3b5eb44e
2. Prover generates t =
27ebf7c9710b5e0b53d0a20dccfcda02c8c6c7f1cd86dd09da20d124b830f1a6
3. Verifier checks: SHA256(27ebf...) -> 79d0... (**MATCH**)
4. Verifier updates vk to
27ebf7c9710b5e0b53d0a20dccfcda02c8c6c7f1cd86dd09da20d124b830f1a6

Test Case 2: Simulation Script

Script Logic:

1. **Setup:** Prover (k ="A", N =5), Verifier initialized with H^6 ("A").
2. **Login 1:** Prover sends token (creates H^5). Verifier accepts.
3. **Login 2:** Prover sends token (creates H^4). Verifier accepts.

4. **Attack Attempt:** Attacker tries sending the *previous* token (H^5) again. Verifier must reject (return False).
5. **Reseed:**
 - Prover creates new chain ($k="B", N=3$).
 - Prover sends ($\text{auth_token}=H^3("A"), \text{new_anchor}=H^4("B")$).
 - Verifier accepts and updates key.
6. **New Login:** Prover sends token ($H^3("B")$). Verifier accepts.

Deliverables

A .py file containing:

1. The Prover and Verifier classes.
2. A `run_test_case_1()` function printing the manual check values.
3. A `run_simulation()` function executing Test Case 2 logic.